Chapter 2 3
Packages 3
Time Definitions 5
Time Values with Time Units 5
Scope-Level Time Unit and Precision 5
Time Unit and Precision Search Order 5
Chapter 3 7
Enhanced Literal Value Assignments 7
SystemVerilog Variables 8
Chapters 4 and 8 9
User-Defined Types 9
Enumerated Types 9
Chapter 5 11
Structures 11
Unions 13
Arrays 13
Unpacked Arrays 13
Packed Arrays 14
Mixed Packed and Unpacked Arrays 14
Using Packed and Unpacked Arrays 15
The Foreach Array Looping Construct 15
Chapter 6 16
Combinational Logic Procedural Blocks 16
Latched Logic Procedural Blocks 17
Sequential Logic Procedural Blocks 17
Enhancements to Tasks and Functions 17
Chapter 7 18
New Operators 18
Operand Enhancements 20

Sign Casting 20
Enhanced for Loops 21
Enhanced Block Names 21
Statement Labels 23
Unique Case Statements 24
Priority Case Statements 25
Enhanced if...else Decisions 26
Unique if...else Decisions 26
Priority if...else Decisions 28
Chapter 9 29
Simplified Netlists of Module Instances 29
Implicit *.name* Port Connection 32
Implicit .* Port Connection 35
Net Aliasing 36
SystemVerilog Port Declarations 39
Chapter 10 40

In this file, I summarize the important points in the “SystemVerilog for Design” book. This book is for using SystemVerilog for design only, there is a complementary book called “SystemVerilog for Verification” for verification. I already highlight the important parts in the book, but this document is more summarized as it focuses exactly on what I need to do while writing RTL files for design.

Chapter 2

Packages

In Verilog, declarations of variables, nets, tasks and functions must be declared within a module, between the ***module***...***endmodule*** keywords. The objects declared within a module are local to the module. Verilog does not have a place to make global declarations; a declaration that is used in multiple design files. For example, a function that is used in many modules must be declared in each module (even if these modules are written in the same file, which is not good coding practice to do).

SystemVerilog adds *packages* to the Verilog language. SystemVerilog packages are defined between the keywords ***package*** and ***endpackage***. The synthesizable constructs that a package can contain are:

- ***parameter*** and ***localparam*** constant definitions.
- ***const*** variable definitions.
- ***typedef*** user-defined types.
- Fully automatic ***task*** and ***function*** definitions.
- ***import*** statements from other packages.
- Operator overload definitions.

In Verilog, a ***parameter*** constant can be redefined for each instance of a module, whereas a ***localparam*** cannot be directly redefined. In a package, however, a ***parameter*** constant cannot be redefined, since it is not part of a module instance. In a package, ***parameter*** and ***localparam*** are synonymous.

The best way (from coding style point of view) to define and use packages is as follows:

1. Define your package(s) in a separate file than your module files with the extension ***.pkg***. You can define each package in a separate file or all related packages together in the same file, as you like.
2. The package file should be as shown below, where you enclose your package's body between ***package*** and ***endpackage***, use the ***wildcard import*** keyword so that all package's content gets imported in all files that include the package, and finally you enclose the file with ***`ifndef ... `endif*** condition to avoid compiling/importing the package multiple times when compiling multiple files that include the package together.

Example 2-6: Package with conditional compilation (file name: `definitions.pkg`)

```
`ifndef DEFS_DONE  // if the already-compiled flag is not set...
  `define DEFS_DONE  // set the flag
  package definitions;

    parameter VERSION = "1.1";

    typedef enum {ADD, SUB, MUL} opcodes_t;

    typedef struct {
      logic [31:0] a, b;
      opcodes_t    opcode;
    } instruction_t;

    function automatic [31:0] multiplier (input [31:0] a, b);
      // code for a custom 32-bit multiplier goes here
      return a * b;  // abstract multiplier (no error detection)
    endfunction
  endpackage

  import definitions::*;  // import package into $unit

`endif
```

3. Include the package at the beginning of every design/testbench file that needs the definitions in the package as shown below.

Example 2-7: A design file that includes the conditionally-compiled package file

```
`include "definitions.pkg" // compile the package file

module ALU
(input  instruction_t  IW,
 input  logic          clock,
 output logic [31:0]   result
);
  always_comb begin
    case (IW.opcode)
      ADD : result = IW.a + IW.b;
      SUB : result = IW.a - IW.b;
      MUL : result = multiplier(IW.a, IW.b);
    endcase
  end
endmodule
```

Notice that ***.pkg*** files do not need to be compiled separately as they are indirectly compiled when included in other design/testbench files.

Time Definitions

Time Values with Time Units

SystemVerilog extends the Verilog language by allowing time units to be specified as part of the time value.

```
forever #5ns clock = ~clock;
```

Notice that no space is allowed between the time value and the time unit. The time units can be any of the following:

Table 2-1: SystemVerilog time units

Unit	Description
s	seconds
ms	milliseconds
us	microseconds
ns	nanoseconds
ps	picoseconds
fs	femtoseconds
step	the smallest unit of time being used by the software tool (used in SystemVerilog testbench clocking blocks)

Scope-Level Time Unit and Precision

SystemVerilog allows the time units and time precision of time values to be specified locally, as part of a module, interface or program block, instead of as commands to the software tool using the keywords ***timeunit*** and ***timeprecision***.

```
module chip (...);
  timeunit 1ns;
  timeprecision 10ps;
      ...
  endmodule
```

Time Unit and Precision Search Order

With SystemVerilog, the time unit and precision of a time value can be specified in multiple places. SystemVerilog defines a specific search order to determine a time value's time unit and precision:

- If specified, use the time unit specified as part of the time value.

- Else, if specified, use the local time unit and precision specified in the module, interface or program block.
- Else, if the module or interface declaration is nested within another module or interface, use the time unit and precision in use by the parent module or interface.
- Else, if specified, use the `timescale time unit and precision in effect when the module was compiled.
- Else, if specified, use the time unit and precision defined in the compilation-unit scope.
- Else, use the simulator's default time unit and precision.

The following example illustrates a mixture of delays. The comments indicate which declaration takes precedence.

Example 2-9: Mixed declarations of time units and precision (not synthesizable)

```
timeunit 1ns;          // external time unit and precision
timeprecision 1ns;

module my_chip ( ... );

  timeprecision 1ps; // local precision (priority over external)

  always @(posedge data_request) begin
    #2.5 send_packet;  // uses external units & local precision
    #3.75ns check_crc; // specific units take precedence
  end

  task send_packet();
    ...
  endtask

  task check_crc();
  ...
  endtask
endmodule

`timescale 1ps/1ps  // directive takes precedence over external
module FSM ( ... );
  timeunit 1ns;     // local units take priority over directive

  always @(State) begin
    #1.2 case (State) // uses local units & timescale precision
      WAITE: #20ps ...;  // specific units take precedence
    ...
  end
endmodule
```

Chapter 3

Enhanced Literal Value Assignments

In the Verilog language, a vector can be easily filled with all zeros, all Xs (unknown), or all Zs (high-impedance).

```
parameter SIZE = 64;
reg [SIZE-1:0] data;

data = 0;   // fills all bits of data with zero
data = 'bz; // fills all bits of data with Z
data = 'bx; // fills all bits of data with X
```

Each of the assignments in the example above is scalable. If the *SIZE* parameter is redefined, perhaps to 128, the assignments will automatically expand to fill the new size of data. However, Verilog does not provide a convenient mechanism to fill a vector with all ones. To specify a literal value with all bits set to one, a fixed size must be specified. For example:

data=64'hFFFFFFFFFFFFFFFF;

This last example is not scalable. If the *SIZE* constant is redefined to a larger size, such as 128, the literal value must be manually changed to reflect the new bit size of *data*. In order to make an assignment of all ones scalable, Verilog designers have had to learn coding tricks, such as using some type of operation to fill a vector with all ones, instead of specifying a literal value. The next two examples illustrate using a ones complement operator and a two's complement operator to fill a vector with all ones:

data = ~0; // one's complement operation

data = -1; // two's complement operation

SystemVerilog enhances assignments of a literal value in two ways. First, a simpler syntax is added, that allows specifying the fill value without having to specify a radix of binary, octal or hexadecimal. Secondly, the fill value can also be a logic 1. The syntax is to specify the value with which to fill each bit, preceded by an apostrophe ('), which is sometimes referred to as a "tick". Thus:

- '0 fills all bits on the left-hand side with 0.
- '1 fills all bits on the left-hand side with 1.
- 'z or 'Z fills all bits on the left-hand side with z.
- 'x or 'X fills all bits on the left-hand side with x.

SystemVerilog Variables

In Verilog, we are used to always using ***reg*** or ***wire*** which are 4-state data types that can take the values 0, 1, x, or z. SystemVerilog adds several new 4-state and 2-state types of which the most important two are ***bit*** and ***logic*** (the other new data types are more suitable for modeling at more abstract levels than RTL, such as system level and transaction level). The only difference between them is that ***bit*** is a 2-state type, while ***logic*** is a 4-state type. From now on, you can dismiss using ***reg*** and ***wire*** and use either ***bit*** or ***logic***. Which one to choose, ***bit*** or ***logic***? Both are commonly used, but ***logic*** is usually preferred since 4-state types are the preferred types for synthesizable RTL hardware models (like ***reg*** and ***wire***) since the *z* value is used to represent unconnected or tri-state design logic, and the *x* value is used to detect and isolate design errors. For example, 4-state variables default to beginning simulation with all bits at a logic X. These variables are considered uninitialized, and, therefore, at an unknown value until a first value is assigned to the variable (for example, by the design reset logic). On the other hand, 2-state data types begin simulation with a logic 0. Since 2-state types do not store an X value, they cannot represent an uninitialized state. This is one of the reasons that it is preferable to use 4-state types to represent synthesizable RTL. But from the synthesis point of view, both ***bit*** and ***logic*** are treated the same.

```
logic resetN; // a 1-bit wide 4-state variable

logic [63:0] data; // a 64-bit wide variable

logic [0:7] array [0:255]; // an array of 8-bit
                              variables

bit resetN; // a 1-bit wide 2-state variable

bit [63:0] data; // a 64-bit 2-state variable

bit [0:7] array [0:255]; // an array of 8-bit
                            2-state variables
```

Chapters 4 and 8

User-Defined Types

SystemVerilog user-defined types are created using the ***typedef*** keyword, as in C. User-defined types allow new type definitions to be created from existing types. Once a new type has been defined, variables of the new type can be declared. For example:

```
typedef int unsigned uint;
...
uint a, b;  // two variables of type uint
```

Usually, we define user-defined types in packages. Also, to make source code easier to read and maintain, a common naming convention is to end all user-defined types with the characters "_t".

Enumerated Types

Enumerated types provide a means to declare an abstract variable that can have a specific list of valid values. Each value is identified with a user-defined name, or *label*. In the following example, variable *RGB* can have the values of *red*, *green* and *blue*:

```
enum {red,green,blue} RGB;
```

In addition to specifying a set of unique labels, SystemVerilog provides two shorthand notations to specify a range of labels in an enumerated type list. The following example creates an enumerated list with the labels *RESET*, *S0* through *S4*, and *W6* through *W9*:

```
enum {RESET, S[5], W[6:9]} state;
```

By default, the actual value represented by the label in an enumerated type list is an integer of the ***int*** type. The first label in the enumerated list is represented with a value of 0, the second label with a value of 1, the third with a value of 2, and so on. SystemVerilog allows the value for each label in the enumerated list to be explicitly declared. This allows the abstract enumerated type to be refined, if needed, to represent more detailed hardware characteristics. For example, a state machine sequence can be explicitly modeled to have one-hot values, one-cold values, Johnson-count, Gray-code, or other type of values. In the next example, the label A is explicitly given a value of 1, B is automatically given the incremented value of 2 and C the incremented value of 3. X is explicitly defined to have a value of 24, and Y and Z are given the incremented values of 25 and 26, respectively.

```
enum {A=1, B, C, X=24, Y, Z} list1;
```

In order to represent hardware at a more detailed level, SystemVerilog allows an explicit base type for the enumerated types to be declared. For example:

```
// enumerated type with a 1-bit wide,
// 2-state base type
enum bit {TRUE, FALSE} Boolean;

// enumerated type with a 2-bit wide,
// 4-state base type
enum logic [1:0] {WAITE, LOAD, READY} state;

 enum logic [2:0] {WAITE = 3'b001,
                   LOAD  = 3'b010,
                   READY = 3'b100} state;
```

The following example shows the case that you will always need ***enum*** in, which is modeling an FSM:

```
module traffic_light (output logic green_light,
                                   yellow_light,
                                   red_light,
                      input        sensor,
                      input [15:0] green_downcnt,
                                   yellow_downcnt,
                      input        clock, resetN);

  enum logic [2:0] {RED    = 3'b001, // explicit enum definition
                    GREEN  = 3'b010,
                    YELLOW = 3'b100} State, Next;

  always_ff @(posedge clock, negedge resetN)
    if (!resetN) State <= RED;  // reset to red light
    else         State <= Next;

  always_comb begin: set_next_state
    Next = State;  // the default for each branch below
    unique case (State)
      RED:    if (sensor)              Next = GREEN;
      GREEN:  if (green_downcnt  == 0) Next = YELLOW;
      YELLOW: if (yellow_downcnt == 0) Next = RED;
    endcase
  end: set_next_state

  always_comb begin: set_outputs
    {green_light, yellow_light, red_light} = 3'b000;
    unique case (State)
      RED:    red_light    = 1'b1;
      GREEN:  green_light  = 1'b1;
      YELLOW: yellow_light = 1'b1;
    endcase
  end: set_outputs
endmodule
```

Chapter 5

Structures

Design data often has logical groups of signals, such as all the control signals for a bus protocol, or all the signals used within a state controller. The Verilog language does not have a convenient mechanism for collecting common signals into a group. Instead, designers must use ad-hoc grouping methods such as naming conventions where each signal in a group starts or ends with a common set of characters. SystemVerilog adds C-like structures to Verilog. A structure is a convenient way of grouping several pieces of related information together. A structure is declared using the struct keyword. Structure members can be any variable type, including user-defined types, and any constant type. An example structure declaration is:

```
struct {
  int          a, b;      // 32-bit variables
  opcode_t opcode;        // user-defined type
  logic [23:0] address;   // 24-bit variable
  bit          error;     // 1-bit 2-state var.
} Instruction_Word;
```

A structure is a collection of variables and/or constants under a single name. The entire collection can be referenced using the name of the structure. Each member within the structure also has a name, which is used to select it from the structure. A structure member is referenced the same as in C.

```
<structure name>.<variable name>
```

For example, to assign a value to the opcode member of the preceding structure, the reference is:

```
Instruction_Word.address = 32'hF000001E;
```

User-defined types can be created from structures, using the ***typedef*** keyword. Declaring a structure as a user-defined type does not allocate any storage. Before values can be stored in the members of a structure that is defined as a user-defined type, a variable of that user-defined type must be declared.

```
typedef struct {  // structure definition
  logic [31:0] a, b;
  logic [ 7:0] opcode;
  logic [23:0] address;
} instruction_word_t;

instruction_word_t IW;  // structure allocation
```

When a structure is declared without using ***typedef***, it is referred to as an *anonymous* structure.

```
struct {
  logic [31:0] a, b;
  logic [ 7:0] opcode;
  logic [23:0] address;
} instruction;
```

The members of a structure can be initialized at the time the structure is instantiated, using a set of values enclosed between the tokens ***'{*** and ***}***. The number of values between the braces must exactly match the number of members in the structure.

```
instruction_word_t IW = '{100, 3, 8'hFF, 0};
```

The previous comma-separated list assignment can be used anywhere, not only during instantiation.

```
always @(posedge clock, negedge resetN)
  if (!resetN) IW = '{100, 5, 8'hFF, 0};
  else begin
    ...
  end
```

The values in the structure expression can be listed in the order in which they are defined in the structure. Alternatively, the structure expression can specify the names of the structure members to which values are being assigned, where the member name and the value are separated by a colon. When member names are specified, the expression list can be in any order.

```
IW = '{address:0, opcode:8'hFF, a:100, b:5};
```

Structures can be passed through module and interface ports. The structure must first be defined as a user-defined type using typedef, which then allows the module or interface port to be declared as the structure type.

```
package definitions;

  typedef enum {ADD, SUB, MULT, DIV} opcode_t;

  typedef struct {
    logic [31:0]  a, b;
    opcode_t      opcode;
    logic [23:0]  address;
    logic         error;
  } instruction_word_t;

endpackage

module alu
(input definitions::instruction_word_t IW,
 input wire                            clock);
  ...
endmodule
```

Unions

I felt unions are useless, so I skipped them.

Arrays

Unpacked Arrays

The syntax of an unpacked array declaration is:

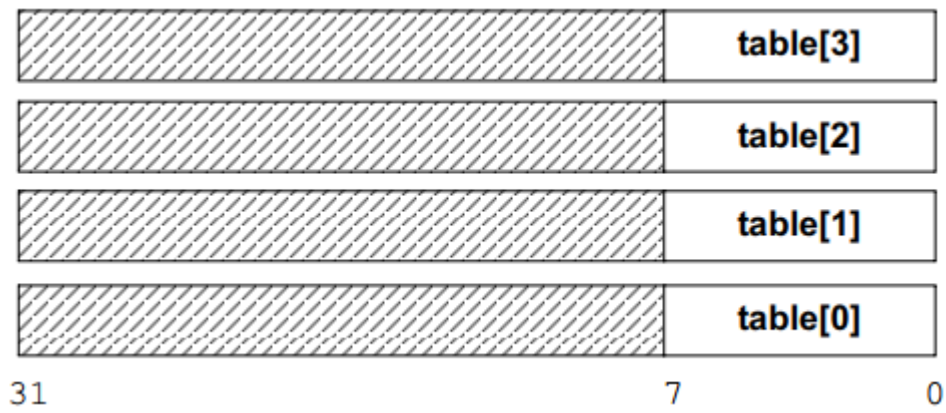
<data type> <array name> <array dimensions>

For example:

***logic** table [3:0] [7:0];*

Notice that an unpacked array dimensions come *after* the signal name.

With unpacked arrays, each element of the array may be stored independently from other elements but grouped under a common array name. Verilog does not define how software tools should store the elements in the array. For example, the following figure illustrates how the previous declaration might be stored within memory.



Packed Arrays

The syntax of a packed array declaration is:

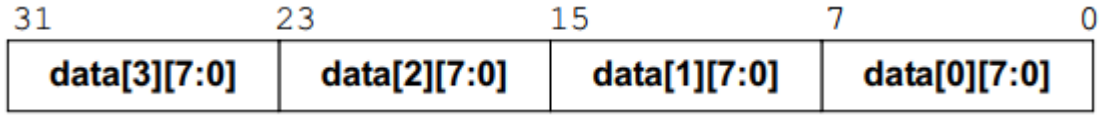
<data type> <array dimensions> <array name>

For example:

logic *[3:0] [7:0] data;*

Notice that a packed array dimensions come *before* the signal name.

SystemVerilog defines how the elements of a packed array are stored. The entire array must be stored as contiguous bits, which is the same as a vector. Each dimension of a packed array is a sub field within the vector. In the packed array declaration above, there is an array of 4 8-bit sub-arrays. The following figure illustrates how the two-dimensional array above will be stored, regardless of the software compiler, operating system or platform.



Mixed Packed and Unpacked Arrays

It is common to have a combination of unpacked arrays and packed arrays. The following example declares an unpacked array of 64-bit packed arrays:

```
logic [63:0] mem [0:4095];
```

When indexing arrays of arrays, unpacked dimensions are referenced first, from the left-most dimension to the right-most dimension. Packed dimensions (vector fields) are referenced second, from the left-most dimension to the right-most dimension. The following figure illustrates the order in which dimensions are selected in a mixed packed and unpacked multi-dimensional array.

```
logic [3:0][7:0] mixed_array [0:7][0:7][0:7];

mixed_array [0] [1] [2] [3] [4] = 1'b1;
```

In Verilog, a packed array is referred to as a vector and is limited to a single dimension. Verilog allows packed arrays to be passed through module ports, or to be passed in or out of tasks and functions. Verilog does not allow unpacked arrays to be passed through module ports, tasks or functions. SystemVerilog extends Verilog by allowing arrays of any type and any number of dimensions to be passed through ports or task/function arguments.

Using Packed and Unpacked Arrays

There are some guidelines in the book on when to use each of them, but there are no mentioned reasons, so I am not convinced. So, in my opinion, always use the mixed packed and unpacked arrays syntax to declare multi-dimensional arrays.

The Foreach Array Looping Construct

SystemVerilog adds a ***foreach*** loop, which can be used to iterate over the elements of single- and multi-dimensional arrays, without having to specify the size of each array dimension. The argument to a ***foreach*** loop is the name of an array followed by a comma-separated list of loop variables enclosed in square brackets. Each loop variable corresponds to one of the dimensions of the array.

```
int sum [1:8] [1:3];

foreach ( sum[i,j] )
  sum[i][j] = i + j;    // initialize array
```

It is not necessary to specify a loop variable for each dimension of an array. A dimension can be skipped by showing a variable position using two commas, without a variable name. Empty loop variables indicate that the loop will not iterate over that dimension of the array. Contiguous empty loop variables at the end of the variable list can be omitted without listing the additional commas.

int *sum [1:8] [1:3][2:8][3:9];*

foreach *(sum[i,,j])*

sum[i][2][j][5] = i + j;

Chapter 6

The Verilog language provides a general-purpose procedural block, called ***always***, that is used to model a variety of hardware types as well as verification routines. The multi-function role of the general purpose ***always*** procedural block places a substantial burden on software tools such as synthesis compilers and formal verification. It is not enough for these types of tools to execute the statements within the procedural block. Synthesis compilers and formal verification tools must also try to deduce what type of hardware is being represented—combinational, latched or sequential logic. To infer the proper type of hardware implementation, synthesis compilers and formal tools must examine the statements and event controls within the procedural block. SystemVerilog adds three specialized procedural blocks to reduce the ambiguity of the Verilog general purpose always procedural block when modeling hardware. These are: ***always_comb***, ***always_latch***, and ***always_ff***. The specialized ***always_comb***, ***always_latch***, and ***always_ff*** procedural blocks indicate the design intent. Software tools do not need to infer from context what the designer intended, as must be done with the general purpose ***always*** procedural block. If the content of a specialized procedural block does not match the rules for that type of logic, software tools can issue warning messages. By using ***always_comb***, ***always_latch***, and ***always_ff*** procedural blocks, the engineer's intent is clearly documented for both software tools and for other engineers who review or maintain the model.

Combinational Logic Procedural Blocks

Unlike the general purpose always procedural block, it is not necessary to specify a sensitivity list with ***always_comb***. A combinational logic sensitivity list can be automatically inferred, because software tools know that the intent is to represent combinational logic.

An important advantage of ***always_comb*** over the general purpose ***always*** procedural block is that when ***always_comb*** is specified, the designer's intent is clearly stated. In the following example with a general-purpose ***always*** procedural block, a software tool cannot know what type of logic the designer intended to represent, and consequently will infer that latched logic was intended, instead of combinational logic.

```
always @(a, en)
  if (en) y = a;
```

With SystemVerilog, this same example could be written as follows:

```
always_comb
  if (en) y = a;
```

Software tools can then tell from the ***always_comb*** keyword that the designer's intent was to model combinational logic and can issue a warning that a latch would be required to realize the procedural block's functionality in hardware.

Latched Logic Procedural Blocks

The ***always_latch*** procedural block is used to indicate that the intent of the procedural block is to model latch-based logic. ***always_latch*** infers its sensitivity list, just like ***always_comb***.

```
always_latch
  if (enable) q <= d;
```

Sequential Logic Procedural Blocks

The ***always_ff*** specialized procedural block indicates that the designer's intent is to model synthesizable sequential logic behavior.

```
always_ff @(posedge clock, negedge resetN)
  if (!resetN) q <= 0;
  else         q <= d;
```

Enhancements to Tasks and Functions

I skipped this part since I don't usually use tasks and functions, but you can read it as SystemVerilog adds a few enhancements to tasks and functions compared to Verilog.

Chapter 7

New Operators

SystemVerilog adds the ***++*** increment operator and the **--** decrement operator to the Verilog language. These operators are used in the same way as in C. For example:

```
for (i = 0; i <= 31; i++ ) begin
  ...
end
```

As in C, the increment and decrement operators can be used to either pre-increment/pre-decrement a variable, or to post-increment/post-decrement a variable. The table below shows the four ways in which the increment and decrement operators can be used.

Statement	Operation	Description
`j = i++;`	post-increment	j is assigned the value of i, and then i is incremented by 1
`j = ++i;`	pre-increment	i is incremented by 1, and j is assigned the value of i
`j = i--;`	post-decrement	j is assigned the value of i, and then i is decremented by 1
`j = --i;`	pre-decrement	i is decremented by 1, and j is assigned the value of i

The increment and decrement operators behave as blocking assignments. The following two statements are semantically equivalent:

```
i++;          // increment i with blocking assign
i = i + 1;    // increment i with blocking assign
```

Just as it is possible to misuse the Verilog blocking assignment, creating a race condition within simulation, it is also possible to misuse the increment and decrement operators. The following example illustrates how an increment or decrement operator could be used in a manner that would create a simulation race condition. In this example, a simple counter is incremented using the ++ operator. The counter, which would be implemented as sequential logic using some form of flip-flops, is modeled using a sequential logic ***always_ff*** procedural block. Another sequential logic procedural block reads the current value of the counter and performs some type of functionality based on the value of the counter.

```
always_ff @(posedge clock)
  if (!resetN) count <= 0;
  else count++;  // same as count = count + 1;

always_ff @(posedge clock)
  case (state)
    HOLD: if (count == MAX)
            ...
```

Since *count++* is equivalent to *count = count + 1*, and we know we should be using *count <= count + 1*, a race condition might occur in the previous example. A non-blocking assignment is required to prevent potential race conditions.

```
always_ff @(posedge clock)
  if (!resetN) count <= 0;
  else count <= count + 1; // nonblocking assign

always_ff @(posedge clock)
  case (state)
    HOLD: if (count == MAX)
            ...
```

The guideline for using ***++*** and **--** is to only use them to model combinational logic. Sequential and latched logic procedural blocks should not use the increment and decrement operators to modify any variables that are to be read outside of the procedural block. Temporary variables that are only read within a sequential or latched logic procedural block can use the ++ and -- operators without race conditions. For example, a variable used to control a ***for*** loop can use the ***++*** or **--** operators even within a sequential procedural block, so long as the variable is not read anywhere outside of the procedural block.

SystemVerilog adds several additional operators shown in the table below.

Operator	Description
+=	add right-hand side to left-hand side and assign
-=	subtract right-hand side from left-hand side and assign
*=	multiply left-hand side by right-hand side and assign
/=	divide left-hand side by right-hand side and assign
%=	divide left-hand side by right-hand side and assign the remainder
&=	bitwise AND right-hand side with left-hand side and assign
|=	bitwise OR right-hand side with left-hand side and assign
^=	bitwise exclusive OR right-hand side with left-hand side and assign
<<=	bitwise left-shift the left-hand side by the number of times indicated by the right-hand side and assign
>>=	bitwise right-shift the left-hand side by the number of times indicated by the right-hand side and assign
<<<=	arithmetic left-shift the left-hand side by the number of times indicated by the right-hand side and assign
>>>=	arithmetic right-shift the left-hand side by the number of times indicated by the right-hand side and assign

The same guidelines apply with these operators; to avoid any race conditions, these operators must be used to model combinational logic only.

SystemVerilog adds an operator to test if a value matches anywhere within a set of values. The operator uses the keyword, ***inside***.

```
logic [2:0] a;

if ( a inside {3'b001, 3'b010, 3'b100} )
   ...
```

The ***inside*** operator can simplify comparing a value to several possibilities. Without the ***inside*** operator, the preceding ***if*** decision would likely have been coded as:

```
if ( (a==3'b001) || (a==3'b010) || (a==3'b100} )
   ...
```

Operand Enhancements

Sign Casting

SystemVerilog also allows explicitly casting the signedness of a value. The syntax for the sign casting operation is:

```
signed'(expression)

unsigned'(expression)
```

Some examples of sign casting are:

```
sum = signed'(a) + signed'(a); // cast operands

if (unsigned'(a-b) <= 5)   // cast intermediate
  ...                      // result
```

The SystemVerilog sign cast operator performs the same conversion as the Verilog ***$signed*** and ***$unsigned*** system functions. Sign casting is synthesizable, following the same rules as the ***$signed*** and ***$unsigned*** system functions.

Enhanced for Loops

With SystemVerilog, the declaration of the ***for*** loop variable can be made within the ***for*** loop itself. This eliminates the need to define several variables at the module level.

```
always_ff @(posedge clock) begin
  for (int i = 1; i <= 1024; i += 1)
      ...
end
```

Enhanced Block Names

Complex code will often have several nested ***begin***...***end*** statement blocks. In such code, it can be difficult to recognize which ***end*** is associated with which ***begin***. The following example illustrates how a single procedural block might contain several nested ***begin***...***end*** blocks. Even with proper indenting and keyword bolding as used in this short example, it can be difficult to see which end belongs with which begin.

```
always_ff @(posedge clock, posedge reset)
  begin
    logic breakVar;
    if (reset) begin
      ... // reset all outputs
    end
    else begin
      case (SquatState)
        wait_rx_valid:
          begin
            Rxready <= '1;
            breakVar = 1;
            for (int j=0; j<NumRx; j+=1) begin
              for (int i=0; i<NumRx; i+=1) begin
                if (Rxvalid[i] && RoundRobin[i] && breakVar)
                  begin
                    ATMcell <= RxATMcell[i];
                    Rxready[i] <= 0;
                  SquatState <= wait_rx_not_valid;
                  breakVar = 0;
                end
            end
          end
        end
      ... // process other SquatState states
    endcase
  end
end
```

Verilog allows a statement block to have a name, by appending :<name> after the ***begin*** keyword. SystemVerilog allows (but does not require) a matching block name after the ***end*** keyword. This additional name does not affect the block semantics in any way but does serve to enhance code readability by documenting which statement group is being completed.

The following code snippet modifies the previous example by adding names to the ***begin***...***end*** statement groups, helping to make the code easier to read.

```
always_ff @(posedge clock, posedge reset)
  begin: FSM_procedure
    logic breakVar;
    if (reset) begin: reset_logic
      ... // reset all outputs
    end: reset logic
  else begin: FSM_sequencer
    unique case (SquatState)
      wait_rx_valid:
        begin: rx_valid_state
          Rxready <= '1;
          breakVar = 1;
          for (int j=0; j<NumRx; j+=1) begin: loop1
            for (int i=0; i<NumRx; i+=1) begin: loop2
              if (Rxvalid[i] && RoundRobin[i] && breakVar)
                begin: match
                  ATMcell <= RxATMcell[i];
                  Rxready[i] <= 0;
                  SquatState <= wait_rx_not_valid;
                  breakVar = 0;
                end: match
            end: loop2
          end: loop1
        end: rx_valid_state
      ... // process other SquatState states
    endcase
  end: FSM_sequencer
end: FSM procedure
```

Statement Labels

In addition to named blocks of statements, SystemVerilog allows a label to be specified before any procedural statement. Statement labels use the same syntax as C:

```
<label> : <statement>
```

A statement label is used to identify a single statement, whereas a named statement block identifies a block of one of more statements.

```
always_comb begin : decode_block
  decoder : case (opcode)
    2'b00:
      outer_loop: for (int i=0; i<=15; i++)
        inner_loop: for (int j=0; j<=15; j++)
           //...
    ... // decode other opcode values
  endcase
end : decode block
```

A ***begin***...***end*** block is a statement and can therefore have either a statement label or a block name.

```
begin: block1    // named block
   ...
end: block1


block2: begin    // labeled block
           ...
        end
```

It is illegal to give a statement block both a label and a block name.

Unique Case Statements

```
module traffic_light (output logic green_light,
                                   yellow_light,
                                   red_light,
                      input        sensor,
                      input [15:0] green_downcnt,
                                   yellow_downcnt,
                      input        clock, resetN);

  enum logic [2:0] {RED    = 3'b001, // explicit enum definition
                    GREEN  = 3'b010,
                    YELLOW = 3'b100} State, Next;

  always_ff @(posedge clock, negedge resetN)
    if (!resetN) State <= RED;  // reset to red light
    else         State <= Next;

  always_comb begin: set_next_state
    Next = State;  // the default for each branch below
    unique case (State)
      RED:    if (sensor)              Next = GREEN;
      GREEN:  if (green_downcnt  == 0) Next = YELLOW;
      YELLOW: if (yellow_downcnt == 0) Next = RED;
    endcase
  end: set_next_state
```

```
  always_comb begin: set_outputs
    {green_light, yellow_light, red_light} = 3'b000;
    unique case (State)
      RED:    red_light    = 1'b1;
      GREEN:  green_light  = 1'b1;
      YELLOW: yellow_light = 1'b1;
    endcase
  end: set_outputs
endmodule
```

The use of the unique modifier to the case statement in the preceding example is important. The ***unique*** modifier to the ***case*** statement specifies three things. First, ***unique case*** specifies that all case selection items can be evaluated in parallel, without priority encoding. Software tools such as synthesis compilers can optimize the decoding logic of the case selection items to create smaller, more efficient implementations. This aspect of ***unique* case** is the same as synthesis *parallel_case* pragma. Second, ***unique case*** specifies that there should be no overlap in the case selection items. During the run-time execution of tools such as simulation, if the value of the case expression satisfies two or more case selection items, a run-time warning will occur. This semantic check can help trap design errors early in the design process. The synthesis *parallel_case* pragma does not provide this important semantic check. Third, ***unique case*** specifies that all values of the case expression that occur during simulation must be covered by the case selection items. With ***unique case***, if a case expression value occurs that does not cause a branch of the case statement to be executed, a runtime warning will occur. This semantic check can also help trap design errors much earlier in the design cycle. This is similar to the *full_case* pragma for synthesis, but the synthesis pragma does not require that other tools perform any checking.

Priority Case Statements

A ***priority case*** statement specifies that:

- At least one case select expression must match the case expression when it is evaluated.
- If more than one case select expression matches the case expression when it is evaluated, the first matching branch must be taken.

The ***priority*** modifier indicates that the designer considers it to be OK for two or more case selection expressions to be true at the same time, and that the order of the case selection items is important. In the following example, the designer has specified that there is priority to the order in which interrupt requests are decoded, with irq0 having the highest priority.

```
always_comb
  priority case (1'b1)
    irq0: irq = 4'b0001;
    irq1: irq = 4'b0010;
    irq2: irq = 4'b0100;
    irq3: irq = 4'b1000;
  endcase
```

Because the model explicitly states that case selection items should be evaluated in order, all software tools must maintain the inferred priority encoding, should it be possible for multiple case selection items to match.

Enhanced if...else Decisions

The SystemVerilog ***unique*** and ***priority*** decision modifiers also work with ***if***...***else*** decisions. These modifiers can also reduce ambiguities with this type of decision and can trap potential design errors early in the modeling phase of a design.

The Verilog ***if***...***else*** statement is often nested to create a series of decisions. For example:

```
logic [2:0] sel;

always_comb begin
  if      (sel == 3'b001) mux_out = a;
  else if (sel == 3'b010) mux_out = b;
  else if (sel == 3'b100) mux_out = c;
end
```

In simulation, a series of ***if***...***else***...***if*** decisions will be evaluated in the order in which the decisions are listed. To maintain the same ordering in hardware implementation, priority encoded logic would be required. Often, however, the specific order is not essential in the desired logic. The order of the decisions is merely the way the engineer happened to list them in the source code.

Unique if...else Decisions

The ***unique*** modifier indicates that the designer's intent is that the order of the decisions is not important. Software tools can optimize the inferred priority of the decision order. For example:

```
logic [2:0] sel;

always_comb begin
  unique if (sel == 3'b001) mux_out = a;
    else if (sel == 3'b010) mux_out = b;
    else if (sel == 3'b100) mux_out = c;
end
```

A ***unique if***...***else*** cannot have overlapping conditions. Software tools will perform checking on a ***unique if*** decision sequence to ensure that all decision conditions in a series of ***if...else...if*** decisions are mutually exclusive. This allows the decision series to be executed in parallel, without priority encoding. A software tool will generate a run-time warning if it determines that more than one condition is true. This warning message can occur at either compile time or run-time. This additional checking can help detect modeling errors early in the verification of the model.

In the following example, there is an overlap in the decision conditions. Any or all the conditions for the first, second and third decisions could be true at the same time. This means that the decisions must be evaluated in the order listed, rather than in parallel. Because the **unique** modifier was specified, software tools can generate a warning that the decision conditions are not mutually exclusive.

```
logic [2:0] sel;

always_comb begin
  unique if (sel[0]) mux_out = a;
    else if (sel[1]) mux_out = b;
    else if (sel[2]) mux_out = c;
end
```

A ***unique if***...***else*** warns of unspecified conditions. When the ***unique*** modifier is specified with an ***if*** decision, software tools are required to generate a run-time warning if the ***if*** statement is evaluated and no branch is executed. The following example would generate a run-time warning if the ***unique if...else...if*** sequence is entered and *sel* has any value other than 1, 2 or 4.

```
always_comb begin
  unique if (sel == 3'b001) mux_out = a;
    else if (sel == 3'b010) mux_out = b;
    else if (sel == 3'b100) mux_out = c;
end
```

This run-time semantic check guarantees that all conditions in the decision sequence that occur during run time have been fully specified. When the decision sequence is evaluated, one branch will be executed. This helps ensure that the logic represented by the decisions can be implemented as combinational logic, without the need for latches.

Priority if...else Decisions

The **priority** modifier indicates that the designer's intent is that the order of the decisions is important. Software tools should maintain the order of the decision sequence. For example:

```
always_comb begin
  priority if (irq0) irq = 4'b0001;
      else if (irq1) irq = 4'b0010;
      else if (irq2) irq = 4'b0100;
      else if (irq3) irq = 4'b1000;
end
```

Because the model explicitly states that the decision sequence above should be evaluated in order, all software tools should maintain the inferred priority encoding. The **priority** modifier ensures consistent behavior from software tools. Simulators, synthesis compilers, equivalence checkers, and formal verification tools can all interpret the decision sequence in the same way.

As with the **unique** modifier, when the **priority** modifier is specified with an **if** decision, software tools will perform run-time checks that a branch is executed each time an **if**...**else**...**if** sequence is evaluated. A run-time warning will be generated if no branch of a **priority if**...**else**...**if** decision sequence is executed. This helps ensure that all conditions in the decision sequence that actually occur during run time have been fully specified, and that when the decision sequences are evaluated, a branch will be executed. The logic represented by the decision sequence can be implemented as priority-encoded combinational logic, without latches.

Chapter 9

Simplified Netlists of Module Instances

A netlist is a list of module instances, with nets connecting the ports of the instances together. Netlists are often generated from software tools, such as synthesis compilers; but netlists are also often defined by hand, such as when connecting design blocks together. Even at the block level, with top-level models, netlists can often be quite large, with a high potential for connection errors that can be difficult to debug. Consider the following example using Verilog's named port connections.

```
module miniPIC (
    inout  wire [7:0] port_a_pins,
    inout  wire [7:0] port_b_pins,
    inout  wire [7:0] port_c_pins,
    input  wire       clk,
    input  wire       resetN
  );

  wire [11:0] instruct_reg, program_data;
  wire [10:0] program_counter, program_address;
  wire [ 7:0] tmr0_reg, status_reg, fsr_reg, w_reg, option_reg,
              reg_file_out, port_a, port_b, port_c, trisa,
              trisb, trisc, data_bus, alu_a, alu_b;
  wire [ 6:0] reg_file_addr;
  wire [ 3:0] alu_opcode;
  wire [ 1:0] alu_a_sel, alu_b_sel;
  wire        reg_file_sel, special_reg_sel, reg_file_enable,
              w_reg_enable, zero_enable, carry_enable, skip,
              isoption, istris, polarity, carry, zero;

  pc_stack pcs ( // module instance with named port connections
    .program_counter(program_counter),
    .program_address(program_address),
    .clk(clk),
    .resetN(resetN),
    .instruct_reg(instruct_reg),
    .data_bus(data_bus),
    .status_reg(status_reg)
  );

  prom prom (
    .dout(program_data),
    .clk(clk),
    .address(program_address)
  );

  instruction_decode decoder (
    .alu_opcode(alu_opcode),
    .alu_a_sel(alu_a_sel),
    .alu_b_sel(alu_b_sel),
    .w_reg_enable(w_reg_enable),
```

```
  .reg_file_sel(reg_file_sel),
  .zero_enable(zero_enable),
  .carry_enable(carry_enable),
  .polarity(polarity),
  .option(isoption),
  .tris(istris),
  .instruct_reg(instruct_reg)
);

register_files regs (
  .dout(reg_file_out),
  .tmr0_reg(tmr0_reg),
  .status_reg(status_reg),
  .fsr_reg(fsr_reg),
  .port_a(port_a),
  .port_b(port_b),
  .port_c(port_c),
  .trisa(trisa),
  .trisb(trisb),
  .trisc(trisc),
  .option_reg(option_reg),
  .w_reg(w_reg),
  .instruct_reg(instruct_reg),
  .program_data(program_data),
  .port_a_pins(port_a_pins),
  .data_bus(data_bus),
  .address(reg_file_addr),
  .clk(clk),
  .resetN(resetN),
  .skip(skip),
  .reg_file_sel(reg_file_sel),
  .zero_enable(zero_enable),
  .carry_enable(carry_enable),
  .w_reg_enable(w_reg_enable),
  .reg_file_enable(reg_file_enable),
  .zero(zero),
  .carry(carry),
  .special_reg_sel(special_reg_sel),
  .isoption(isoption),
  .istris(istris)
);

alu alu (
  .y(data_bus),
  .carry_out(carry),
  .zero_out(zero),
  .a(alu_a),
  .b(alu_b),
```

```
    .opcode(alu_opcode),
    .carry_in(status_reg[0])
  );

  glue_logic glue (
    .port_b_pins(port_b_pins),
    .port_c_pins(port_c_pins),
    .alu_a(alu_a),
    .alu_b(alu_b),
    .expan_out(expan_out),
    .expan_addr(expan_addr),
    .reg_file_addr(reg_file_addr),
    .reg_file_enable(reg_file_enable),
    .special_reg_sel(special_reg_sel),
    .expan_read(expan_read),
    .expan_write(expan_write),
    .skip(skip),
    .instruct_reg(instruct_reg),
    .program_counter(program_counter),
    .port_a(port_a),
    .port_b(port_b),
    .port_c(port_c),
    .data_bus(data_bus),
    .expan_in(expan_in),
    .fsr_reg(fsr_reg),
    .tmr0_reg(tmr0_reg),
    .status_reg(status_reg),
    .w_reg(w_reg),
    .reg_file_out(reg_file_out),
    .alu_a_sel(alu_a_sel),
    .alu_b_sel(alu_b_sel),
    .reg_file_sel(reg_file_sel),
    .polarity(polarity),
    .zero(zero)
  );
endmodule
```

The disadvantage of the named port connection style is that it is very verbose. Netlists can contain tens or hundreds of module instances, and each instance can have dozens of ports. Both the name of the port and the name of the net connected to the port must be listed for each and every port connection in the netlist. Port and net names can be up to 1024 characters long in Verilog tools. When long, descriptive port names and net names are used, and there are many ports for each module name, the size and verbosity of a netlist using named port connections can become excessively large and difficult to maintain.

Implicit *.name* Port Connection

SystemVerilog provides three enhancements that greatly simplify netlists: .name (pronounced "*dot-name*") port connections, .* (pronounced "*dot-star*") port connections, and *interfaces*. The .name and .* styles are discussed in the following subsections, and interfaces are presented in Chapter 10.

In many Verilog netlists, especially top-level netlists that connect major design blocks together, it is common to use the same name for both the port name and the name of the net connected to the port. For example, the module might have a port called *data*, and the interconnected net is also called *data*. Using Verilog's named port connection style, it is necessary to repeat the name twice in order to connect the net to the port, for example: *.data(data)*. SystemVerilog simplifies the named port connection syntax by allowing just the port name to be specified. When only the port name is given, SystemVerilog infers that a net or variable of the same name will automatically be connected to the port. This means the verbose Verilog style of *.data(data)* can be reduced to simply *.data*.

The following example simplies the example shown previously using SystemVerilog's *.name* port conection styles for all nets that are the same name as the port.

```
module miniPIC (
    inout  wire [7:0] port_a_pins,
    inout  wire [7:0] port_b_pins,
    inout  wire [7:0] port_c_pins,
    input  wire       clk,
    input  wire       resetN
  );

  wire [11:0] instruct_reg, program_data;
  wire [10:0] program_counter, program_address;
  wire [ 7:0] tmr0_reg, status_reg, fsr_reg, w_reg, option_reg,
              reg_file_out, port_a, port_b, port_c, trisa,
              trisb, trisc, data_bus, alu_a, alu_b;
  wire [ 6:0] reg_file_addr;
  wire [ 3:0] alu_opcode;
  wire [ 1:0] alu_a_sel, alu_b_sel;
  wire        reg_file_sel, special_reg_sel, reg_file_enable,
              w_reg_enable, zero_enable, carry_enable, skip,
              isoption, istris, polarity, carry, zero;

  pc_stack pcs (  // module instance with .name port connections
    .program_counter,
    .program_address,
    .clk,
    .resetN,
    .instruct_reg,
    .data_bus,
```

```
  .status_reg
);

prom prom (
  .dout(program_data),
  .clk,
  .address(program_address)
);

instruction_decode decoder (
  .alu_opcode,
  .alu_a_sel,
  .alu_b_sel,
  .w_reg_enable,
  .reg_file_sel,
  .zero_enable,
  .carry_enable,
  .polarity,
  .option(isoption),
  .tris(istris),
  .instruct_reg
);

register_files regs (
  .dout(reg_file_out),
  .tmr0_reg,
  .status_reg,
  .fsr_reg,
  .port_a,
  .port_b,
  .port_c,
  .trisa,
  .trisb,
  .trisc,
  .option_reg,
  .w_reg,
  .instruct_reg,
  .program_data,
  .port_a_pins,
.data_bus,
.address(reg_file_addr),
.clk,
.resetN,
.skip,
.reg_file_sel,
.zero_enable,
.carry_enable,
.w_reg_enable,
```

```
  .reg_file_enable,
  .zero,
  .carry,
  .special_reg_sel,
  .isoption,
  .istris
);

alu alu (
  .y(data_bus),
  .carry_out(carry),
  .zero_out(zero),
  .a(alu_a),
  .b(alu_b),
  .opcode(alu_opcode),
  .carry_in(status_reg[0])
);

glue_logic glue (
  .port_b_pins,
  .port_c_pins,
  .alu_a,
  .alu_b,
  .reg_file_addr,
  .reg_file_enable,
  .special_reg_sel,
  .skip,
  .instruct_reg,
  .program_counter,
  .port_a,
  .port_b,
  .port_c,
  .data_bus,
  .fsr_reg,
  .tmr0_reg,
  .status_reg,
  .w_reg,
  .reg_file_out,
  .alu_a_sel,
    .alu_b_sel,
    .reg_file_sel,
    .polarity,
    .zero
  );
endmodule
```

Implicit .* Port Connection

SystemVerilog provides an additional short cut to simplify the specification of large netlists. The **.*** syntax indicates that all ports and nets (or variables) of the same name should automatically be connected for that module instance. As with the .name syntax, for a connection to be inferred, the name and vector size must match exactly, and the types connected must be compatible. Any connections that cannot be inferred by .* must be explicitly connected using Verilog's named port connection syntax. The following example illustrates this:

```
module miniPIC (
    inout  wire [7:0] port_a_pins,
    inout  wire [7:0] port_b_pins,
    inout  wire [7:0] port_c_pins,
    input  wire       clk,
    input  wire       resetN
  );

  wire [11:0] instruct_reg, program_data;
  wire [10:0] program_counter, program_address;
  wire [ 7:0] tmr0_reg, status_reg, fsr_reg, w_reg, option_reg,
              reg_file_out, port_a, port_b, port_c, trisa,
              trisb, trisc, data_bus, alu_a, alu_b;
  wire [ 6:0] reg_file_addr;
  wire [ 3:0] alu_opcode;
  wire [ 1:0] alu_a_sel, alu_b_sel;
  wire        reg_file_sel, special_reg_sel, reg_file_enable,
              w_reg_enable, zero_enable, carry_enable, skip,
              isoption, istris, polarity, carry, zero;


  pc_stack pcs (  // module instance with .* port connections
    .*
  );

  prom prom (
    .*,
    .dout(program_data),
    .address(program_address)
  );

  instruction_decode decoder (
    .*,
    .option(isoption),
    .tris(istris)
  );

  register_files regs (
    .*,
    .dout(reg_file_out),
    .address(reg_file_addr)
    );
```

```
alu alu (
  .y(data_bus),
  .carry_out(carry),
  .zero_out(zero),
  .a(alu_a),
    .b(alu_b),
    .opcode(alu_opcode),
    .carry_in(status_reg[0])
  );
  glue_logic glue (
    .*
  );
endmodule
```

Net Aliasing

SystemVerilog adds an **alias** statement that allows two different names to reference the same net. For example:

```
wire clock;
wire clk;

alias clk = clock;
```

The net clk is an alias for clock, and clock is an alias for clk. Both names refer to the same logical net. Several nets can be aliased together. A change on any of the net names will be reflected on all of the nets that are aliased together.

```
wire reset, rst, resetN, rstN;

alias rst = reset;
alias reset = resetN;
alias resetN = rstN;
```

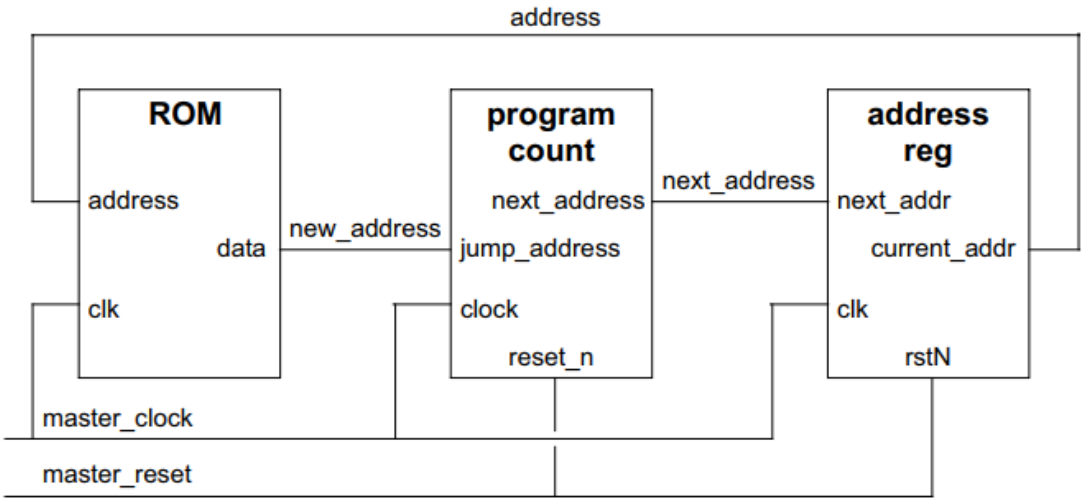
The previous set of aliases can also be abbreviated to a single statement containing a series of aliases, as follows:

```
alias rst = reset = resetN = rstN;
```

The order in which nets are listed in an alias statement does not matter.

SystemVerilog imposes several restrictions on what signals can be aliased to another name. The most important rule is that only the net types can be aliased. Variables cannot be aliased. This means only ***wire*** can be used in aliasing, ***logic*** cannot be used.

The alias statement enables greater usage of the .name and .* shortcuts for modeling netlists. In the following example, however, these shortcuts cannot be fully utilized to connect the clock signals together, because the port names are not the same in each of the modules.




```
module chip (input wire master_clock,
             input wire master_reset,
             ...);

  wire [31:0] address, new_address, next_address;

  ROM           i1 ( .*,  // infers .address(address)
                     .data(new_address),
                     .clk(master_clock) );

  program_count i2 ( .*,  // infers .next_address(next_address)
                     .jump_address(new_address),
                     .clock(master_clock),
                     .reset_n(master_reset) );

  address_reg   i3 ( .*,  // no connections can be inferred
                     .next_addr(next_address),
                     .current_addr(address),
                     .clk(master_clock),
                     .rstN(master_reset) );

endmodule

module ROM (output wire [31:0] data,
            input  wire [31:0] address,
            input  wire        clk);
  ...
endmodule
```

```
module program_count (output logic [31:0] next_address,
                      input  wire  [31:0] jump_address,
                      input  wire         clock, reset_n);
  ...
endmodule

module address_reg (output wire [31:0] current_addr,
                    input  wire [31:0] next_addr,
                    input  wire        clk, rstN);
  ...
endmodule
```

The following example adds alias statements, which allow the netlist to take full advantage of the .* shortcut to connect all modules together.

```
module chip (input wire master_clock,
             input wire master_reset,
             ...);

  wire [31:0] address, data, new_address, jump_address,
              next_address, next_addr, current_addr;

  alias clk = clock = master_clock;
  alias rstN = reset_n = master_reset;
  alias data = new_address = jump_address;
  alias next_address = next_addr;
  alias current_addr = address;

  ROM           i1 ( .* );
```

```
  program_count i2 ( .* );
  address_reg   i3 ( .* );

endmodule

module ROM (output wire [31:0] data,
            input  wire [31:0] address,
            input  wire        clk);
  ...
endmodule

module program_count (output logic [31:0] next_address,
                      input  wire  [31:0] new_count,
                      input  wire         clock, reset_n);
  ...
endmodule

module address_reg (output wire [31:0] address,
                    input  wire [31:0] next_address,
                    input  wire        clk, rstN);
  ...
endmodule
```

SystemVerilog Port Declarations

SystemVerilog simplifies the declaration of module ports in several ways:

- SystemVerilog specifies a default port direction of **inout** (bidirectional). Therefore, it is no longer required to specify a port direction, unless the direction is different than the default.
- if the next port in the port list has a type defined, but no direction is specified, the direction defaults to the direction of the previous port in the list.

An example is shown below:

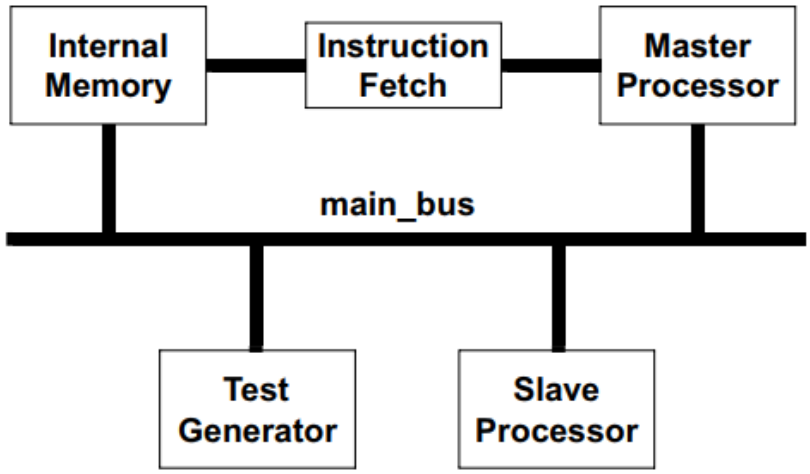
```
module accum (wire [31:0]  data,
              output reg [31:0] result, reg co,
              input [31:0] a, b, tri1 ci );
  ...
endmodule
```

The first port in the list, data, has a type, but no explicit port direction. Therefore, this port defaults to the direction of **inout**. Port co also has a type, but no port direction. This port defaults to the direction of the previous port in the list, which is **output**. Ports a and b have a port direction declared, but no type. As with Verilog-2001 and Verilog-1995, an implicit net type will be inferred, which by default is the type **wire**. Finally, port ci has a type declared, but no port direction. This port will inherit the direction of the previous port in the list, which is **input**.

Chapter 10

Consider the following example that connects five blocks of a design together using a rudimentary bus architecture called *main_bus*, plus some additional connections between some of the design blocks. Figure 10-1 shows the block diagram for this simple design, and example 10-1 lists the Verilog source code for the module declarations involved.

Figure 10-1: Block diagram of a simple design



Example 10-1: Verilog module interconnections for a simple design

```
/********************** Top-level Netlist ********************/
module top (input wire clock, resetN, test_mode);
  wire [15:0] data, address, program_address, jump_address;
  wire [ 7:0] instruction, next_instruction;
  wire [ 3:0] slave_instruction;
  wire        slave_request, slave_ready;
  wire        bus_request, bus_grant;
  wire        mem_read, mem_write;
  wire        data_ready;
```

```
processor proc1 (
  // main_bus ports
  .data(data),
  .address(address),
  .slave_instruction(slave_instruction),
  .slave_request(slave_request),
  .bus_grant(bus_grant),
  .mem_read(mem_read),
  .mem_write(mem_write),
  .bus_request(bus_request),
  .slave_ready(slave_ready),
  // other ports
  .jump_address(jump_address),
  .instruction(instruction),
  .clock(clock),
  .resetN(resetN),
  .test_mode(test_mode)
);
```

signals for main_bus must be individually connected to each module instance

```
slave slave1 (
  // main_bus ports
  .data(data),
  .address(address),
  .bus_request(bus_request),
  .slave_ready(slave_ready),
  .mem_read(mem_read),
  .mem_write(mem_write),
  .slave_instruction(slave_instruction),
  .slave_request(slave_request),
  .bus_grant(bus_grant),
  .data_ready(data_ready),
  // other ports
  .clock(clock),
  .resetN(resetN)
);
```

main_bus connections

```
dual_port_ram ram (
  // main_bus ports
  .data(data),
  .data_ready(data_ready),
  .address(address),
  .mem_read(mem_read),
  .mem_write(mem_write),
  // other ports
  .program_address(program_address),
  .data_b(next_instruction)
);
```

main_bus connections

```
    test_generator test_gen(
    // main_bus ports
    .data(data),
    .address(address),
    .mem_read(mem_read),
    .mem_write(mem_write),
    // other ports
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );

  instruction_reg ir (
    .program_address(program_address),
    .instruction(instruction),
    .jump_address(jump_address),
    .next_instruction(next_instruction),
    .clock(clock),
    .resetN(resetN)
  );
endmodule
```

main_bus connections

```
/******************** Module Definitions ********************/
module processor (
    // main_bus ports
    inout  wire [15:0] data,
    output reg  [15:0] address,
    output reg  [ 3:0] slave_instruction,
    output reg         slave_request,
    output reg         bus_grant,
    output wire        mem_read,
    output wire        mem_write,
    input  wire        bus_request,
    input  wire        slave_ready,
    // other ports
    output reg  [15:0] jump_address,
    input  wire [ 7:0] instruction,
    input  wire        clock,
   input  wire         resetN,
   input  wire         test_mode
 );
 ...  // module functionality code
endmodule
```

ports for main_bus must be individually declared in each module definition

```
module slave (
    // main_bus ports
    inout  wire [15:0] data,
    inout  wire [15:0] address,
    output reg         bus_request,
    output reg         slave_ready,
    output wire        mem_read,
    output wire        mem_write,
    input  wire [ 3:0] slave_instruction,
    input  wire        slave_request,
    input  wire        bus_grant,
    input  wire        data_ready,
    // other ports
    input  wire        clock,
    input  wire        resetN
  );
  ...  // module functionality code
endmodule
```

main_bus port declarations

```
module dual_port_ram (
    // main_bus ports
    inout  wire [15:0] data,
    output wire        data_ready,
    input  wire [15:0] address,
    input  tri0        mem_read,
    input  tri0        mem_write,
    // other ports
    input  wire [15:0] program_address,
    output reg  [ 7:0] data_b
  );
  ...  // module functionality code
endmodule
```

main_bus port declarations

```
module test_generator (
    // main_bus ports
    output wire [15:0] data,
    output reg  [15:0] address,
    output reg         mem_read,
    output reg         mem_write,
    // other ports
    input  wire        clock,
    input  wire        resetN,
    input  wire        test_mode
  );
  ...  // module functionality code
endmodule
```

main_bus port declarations

```
module instruction_reg (
    output reg  [15:0] program_address,
    output reg  [ 7:0] instruction,
    input  wire [15:0] jump_address,
    input  wire [ 7:0] next_instruction,
    input  wire        clock,
    input  wire        resetN
  );
  ...  // module functionality code
endmodule
```

One disadvantage of using Verilog's module ports to connect major blocks of a design together is readily apparent in the example code above. The signals that make up main_bus in the preceding example must be declared in each module that uses the bus, as well as in the top-level netlist that connects the design together. In this simple example, there are only a handful of signals in main_bus, so the redundant declarations are mostly just an inconvenience. In a large, complex design, however, this redundancy becomes much more than an inconvenience. A large design could have dozens of modules connected to the same bus, with dozens of duplicated declarations in each module.

SystemVerilog adds a powerful new port type to Verilog, called an ***interface***. An interface allows a number of signals to be grouped together and represented as a single port. The declarations of the signals that make up the interface are contained in a single location. Each module that uses these signals then has a single port of the interface type, instead of many ports with the discrete signals.

Example 10-2 shows how SystemVerilog's interfaces can reduce the amount of code required to model the simple design shown in Figure 10-1. By encapsulating the signals that make up main_bus as an interface, the redundant declarations for these signals within each module are eliminated.

Example 10-2: SystemVerilog module interconnections using interfaces

```
/****************** Interface Definitions ******************/
interface main_bus;
  wire  [15:0] data;
  wire  [15:0] address;
  logic [ 7:0] slave_instruction;
  logic        slave_request;
  logic        bus_grant;
  logic        bus_request;
  logic        slave_ready;
  logic        data_ready;
  logic        mem_read;
  logic        mem_write;
endinterface
```

signals for main_bus are defined in just one place

```
/********************** Top-level Netlist ******************/
module top (input logic clock, resetN, test_mode);
  logic [15:0] program_address, jump_address;
  logic [ 7:0] instruction, next_instruction;

  main_bus bus ( );  // instance of an interface
                     // (instance name is bus)

  processor proc1 (
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
    .jump_address(jump_address),
    .instruction(instruction),
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );
```

each module instance has a single connection for main_bus

```
  slave slave1 (
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
    .clock(clock),
    .resetN(resetN)
  );
```

main_bus connections

```
  dual_port_ram ram (
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
```

main_bus connections

```
    .program_address(program_address),
    .data_b(next_instruction)
  );

    test_generator test_gen(
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );

  instruction_reg ir (
    .program_address(program_address),
    .instruction(instruction),
    .jump_address(jump_address),
    .next_instruction(next_instruction),
    .clock(clock),
    .resetN(resetN)
  );
endmodule
```

main_bus connections

```
/******************** Module Definitions ********************/
module processor (
    // main_bus interface port
    main_bus bus,  // interface port
    // other ports
    output logic [15:0] jump_address,
    input  logic [ 7:0] instruction,
    input  logic        clock,
    input  logic        resetN,
    input  logic        test_mode
  );
  ...  // module functionality code
endmodule
```

each module definition has a single port for main_bus

```
module slave (
    // main_bus interface port
    main_bus bus,  // interface port
    // other ports
    input  logic        clock,
    input  logic        resetN
  );
  ...  // module functionality code
endmodule
```

main_bus port declaration

```
module dual_port_ram (
    // main_bus interface port
    main_bus bus,  // interface port
    // other ports
    input  logic [15:0] program_address,
    output logic [ 7:0] data_b
  );
  ...  // module functionality code
endmodule
```

main_bus port declaration

```
module test_generator (
    // main_bus interface port
    main_bus bus,  // interface port
    // other ports
    input  logic        clock,
    input  logic        resetN,
    input  logic        test_mode
  );
  ...  // module functionality code
endmodule
```

main_bus port declaration

```
module instruction_reg (
    output logic [15:0] program_address,
    output logic [ 7:0] instruction,
    input  logic [15:0] jump_address,
    input  logic [ 7:0] next_instruction,
    input  logic        clock,
    input  logic        resetN
  );
  ...  // module functionality code
endmodule
```

Syntactically, the definition of an interface is very similar to the definition of a module. An interface can have ports, just as a module does. This allows signals that are external to the interface, such as a clock or reset line, to be brought into the interface and become part of the bundle of signals represented by the interface. Example 10-3 shows a definition for an interface called *main_bus*, with three external signals coming into the interface: *clock, resetN* and *test_mode*. These external signals can now be connected to each module through the interface, without having to explicitly connect the signals to each module.

Example 10-3: The interface definition for `main_bus`, with external inputs

```
/******************* Interface Definitions *******************/
interface main_bus (input logic clock, resetN, test_mode);
  wire  [15:0] data;
  wire  [15:0] address;
  logic [ 7:0] slave_instruction;
  logic        slave_request;
  logic        bus_grant;
  logic        bus_request;
  logic        slave_ready;
  logic        data_ready;
  logic        mem_read;
  logic        mem_write;
endinterface

/********************** Top-level Netlist ********************/
module top (input logic clock, resetN, test_mode);
  logic [15:0] program_address, jump_address;
  logic [ 7:0] instruction, next_instruction;

  main_bus bus (    // instance of an interface
    .clock(clock),
    .resetN(resetN),
    .test_mode(test_mode)
  );

  processor proc1 (
    // main_bus ports
    .bus(bus),  // interface connection
    // other ports
    .jump_address(jump_address),
    .instruction(instruction)
  );

  ...

/*** remainder of netlist and module definitions are not  ***/
/*** listed – they are similar to example 10-2, but       ***/
/*** clock and resetN do not need to be passed to each    ***/
/*** module instance as discrete ports.                    ***/
```

discrete signals are inputs to the interface

discrete signals are connected to the interface instance

discrete signals do not need to be connected to each design block instance

The SystemVerilog simplified port connection styles of *.name* and *.** can also be used with interface port connections.

Example 10-4: Using interfaces with .* connections to simplify complex netlists

```
/****************** Interface Definitions ******************/
interface main_bus (input logic clock, resetN, test_mode);
  wire  [15:0] data;
  wire  [15:0] address;
  logic [ 7:0] slave_instruction;
  logic        slave_request;
  logic        bus_grant;
  logic        bus_request;
  logic        slave_ready;
  logic        data_ready;
  logic        mem_read;
  logic        mem_write;
endinterface

/********************** Top-level Netlist ********************/
module top (input logic clock, resetN, test_mode);
  logic [15:0] program_address, jump_address;
  logic [ 7:0] instruction, next_instruction, data_b;

  main_bus        bus      ( .* );
  processor       proc1    ( .* );
  slave           slave1   ( .* );
  instruction_reg ir       ( .* );
  test_generator  test_gen ( .* );
  dual_port_ram   ram      ( .*, .data_b(next_instruction) );

endmodule

/*** remainder of netlist and module definitions are not  ***/
/*** listed — they are similar to example 10-2, but       ***/
/*** clock and resetN do not need to be passed to each    ***/
/*** module instance as discrete ports.                    ***/
```

.* port connections can significantly reduce a netlist (compare to netlist in example 10-2 on page 270).

A port can be declared using the ***interface*** keyword to define the port as a generic interface. When the module is instantiated, any interface can be connected to the generic interface port.

```
module RAM (interface pins,
            input     clock);
  ...
endmodule
```